

Towards Reliable Broadcast with Optimal Communication and Round Complexity

Thomas Locher¹ Victor Shoup² 

¹ DFINITY

thomas.locher@dfinity.org

² Offchain Labs

victor@shoup.net

October 3, 2025

Abstract. The reliable broadcast protocol with the best communication complexity for long messages to date is the MiniCast protocol of Locher & Shoup (2024). To reliably broadcast a message m to n parties, MiniCast has communication complexity $\sim 1.5|m|n$ when the size $|m|$ of m is large. However, the round complexity of MiniCast is 4, which is worse than the 3 rounds of the classical protocol of Bracha. We give a new reliable broadcast protocol whose communication complexity is essentially the same as that of MiniCast, but whose round complexity is just 3. For large $|m|$, the communication complexity of our new protocol is essentially optimal (for 3-round protocols with somewhat balanced communication). Like MiniCast, our new protocol does not rely on any cryptography other than hash functions. We also give two new 2-round protocols that rely on signatures. The communication complexity of the first protocol is also $\sim 1.5|m|n$, unless the sender provably misbehaves, in which case its communication complexity may degrade to $\sim 2|m|n$. The communication complexity of the second protocol is $\sim 1.5|m|n$, even in the worst case, but (unlike our other protocols) the communication is unbalanced.

1 Introduction

A **reliable broadcast** protocol allows a sender S to broadcast a single message m to parties $P_1, \dots, P_n$ (where S is one of these parties). We assume at most t of the n parties are corrupt, where $t < n/3$, and that the parties are connected by authenticated point-to-point channels. The communication network is completely asynchronous (i.e., there is no bound on the time it takes to deliver messages) and the scheduling of message delivery is entirely under the control of an adversary. Such a protocol should satisfy the following **correctness** property (informally stated):

All honest parties that output a message must output the same message. Moreover, if the sender S is honest, that message is the one input by S .

It should also satisfy the following **completeness** property (informally stated):

If any honest party outputs a message or an honest sender S inputs a message, then eventually all honest parties output a message.

Here, “eventually” means if and when all protocol messages sent from honest parties to honest parties have been delivered.

The problem of reliable broadcast was first studied by Bracha [Bra87], who also gave a simple protocol for it. Reliable broadcast has proved to be a fundamental building block in many other protocols, too numerous to mention here.

An important metric in evaluating any reliable broadcast protocol is its *communication complexity*, which is defined as the total number of bits transmitted by all honest parties over the network in the worst case. Bracha’s protocol has a communication complexity of $O(|m|n^2)$, where $|m|$ is the length (in bits) of the message m to be broadcast—essentially, every party echoes m to every other party. For large $|m|$, this communication complexity can be impractical.

Another important metric is the *round complexity* of the protocol, which we define as the number of rounds of communication required to complete the protocol when the sender is honest. Bracha’s protocol has a round complexity of 3.

For long messages, the protocol with the best communication complexity is the MiniCast protocol [LS25]. The MiniCast protocol has a communication complexity of

$$1.5|m|n + O(\kappa n^2 \log(n)),$$

where κ is the output length of a hash function. Note that the term $O(\kappa n^2 \log(n))$ becomes irrelevant when $|m|$ is sufficiently large (when $|m| \gg \kappa n \log(n)$). Indeed, the communication complexity is $\sim 1.5|m|n$ as $|m| \rightarrow \infty$. Moreover, the communication of MiniCast is well balanced—it can be implemented so that every party, *including the sender*, transmits (and receives) $\sim 1.5|m|$ bits. This balance property can be important in asymmetric applications where just one party broadcasts in each stage of the protocol (such as leader-based atomic broadcast protocols, like PBFT [CL99] and its many descendants), and where communication bandwidth is limited. Unfortunately, the round complexity of MiniCast is 4, which is worse than the round complexity of Bracha’s protocol.

In this paper, we present a variation of MiniCast, called MiniCast*, which achieves essentially the same communication complexity as MiniCast, but with a round complexity of just 3, matching the round complexity of Bracha’s protocol. While the communication of MiniCast* is somewhat balanced, it is not quite as balanced as that of MiniCast: the sender transmits $\sim 4.5|m|$ bits, while every other party transmits (and receives) $\sim 1.5|m|$ bits. This modest imbalance may potentially reduce the usefulness of MiniCast* in asymmetric applications. However, in symmetric applications where all parties simultaneously broadcast in each stage of the protocol this imbalance has no practical impact. Examples of such applications include DAG-based atomic broadcast protocols, like DAG-rider [KKNS21] and Narwhal&Tusk [DKSS22], as well as other leaderless atomic broadcast protocols, such as HoneyBadgerBFT [MXC⁺16].

For large $|m|$, the communication complexity of MiniCast* is essentially optimal. Indeed, as shown in [Loc24], any protocol with a round complexity of 3 and with somewhat balanced communication must have a communication complexity of at least $\sim 1.5|m|n$.

Protocol MiniCast* relies on no cryptography other than hash functions. Using signatures, it is possible to design a reliable broadcast with a round complexity of just 2, as was shown in [ANRX21]. We give two such protocols.

The first is an “optimistic” variation of MiniCast*, called OptMiniCast*. The communication complexity of OptMiniCast* is also $\sim 1.5|m|n$, unless the sender *provably* misbehaves, in which case its communication complexity may degrade to $\sim 2|m|n$. Moreover, the communication of OptMiniCast* is also somewhat balanced: the sender transmits $\sim 6|m|$ bits, while every other party transmits (and receives) $\sim 1.5|m|$ bits, unless the sender provably misbehaves, in which case some parties may transmit $\sim 3|m|$ bits (and receive $\sim 2.5|m|$ bits).

The second is an “unbalanced” version of the original MiniCast protocol, called UB-MiniCast. Unlike OptMiniCast*, the UB-MiniCast protocol has a *worst-case* communication complexity of $\sim 1.5|m|n$. Moreover, the communication of UB-MiniCast is completely unbalanced, with the sender

transmitting $\sim |m|n$ bits, while every other party transmits at most $\sim 0.5|m|$ bits. As mentioned above, in symmetric applications, such an imbalance has no practical impact. In fact, for one variant of UB-MiniCast, in the “happy case”, where the sender is honest, and there are no faults (or just a few faults) among the other parties, and the network is well behaved, the protocol’s communication complexity is $\sim |m|n$, adding essentially no communication overhead to the trivial (insecure) broadcast protocol in which the sender just sends its message to all parties. As we shall discuss, despite the better worst-case communication complexity of UB-MiniCast, there are still practical tradeoffs to be made in choosing between OptMiniCast* over UB-MiniCast.

We stress that while all of the protocols we present here rely heavily on erasure codes, the use of such codes should not, in practice, add significant overhead to the protocols. Indeed, based on reported benchmarks [LS25], well-engineered implementations³ of modern algorithms [LAHC16, LC12] can encode and decode at rates faster than typical network speeds.

1.1 Related work

As already mentioned, Bracha’s reliable broadcast protocol has a communication complexity of $O(|m|n^2)$ and a round complexity of 3. Bracha’s protocol uses no cryptography. As was shown in [ANRX21], using signatures, one can reduce the round complexity of Bracha’s protocol from 3 to 2, while maintaining the same communication complexity (as $|m| \rightarrow \infty$).

The first significant improvement to the communication complexity of reliable broadcast was made by Cachin and Tessaro [CT05], who gave a protocol with communication complexity $O(|m|n + \kappa n^2 \log(n))$, where, again, κ is the output length of a hash function. Note that the security of their protocol relies on the collision resistance of this hash function (unlike Bracha’s protocol, which makes no other assumptions beyond authenticated channels). Their protocol can be implemented so that it attains a (well-balanced) communication complexity of $\sim 3|m|n$ (as $|m| \rightarrow \infty$) and a round complexity of 3. See, for example, the discussion in [ADVZ22], as well as [SS24] for a detailed description of this implementation, and Section 4.3 of [LS25] (or Section 4.1.2 of the full version) for a simple “balancing technique” that makes it truly well balanced by reducing the amount of data transmitted by the sender.

As was shown in [CDH⁺22], using signatures along the same lines as in [ANRX21], one can reduce the round complexity of Cachin and Tessaro’s protocol from 3 to 2, while maintaining the same communication complexity (as $|m| \rightarrow \infty$).

Many other papers have been written on reducing the term $O(\kappa n^2 \log(n))$ in the communication complexity of Cachin and Tessaro’s protocol (see again [ADVZ22] for a discussion). However, this line of research does not lead to any practical improvement for large $|m|$. Indeed, for large $|m|$, the interesting question is to reduce the constant 3 in the term $3|m|n$. This constant arises from the erasure coding technique used in these protocols. The message m is encoded as a vector $f_1, \dots, f_n$ of “fragments”, from any $n - 2t$ of which it is possible to reconstruct m . The reason for the reconstruction threshold of $n - 2t$ is a bit technical. Roughly speaking, in these protocols, the sender gives each party P_i its corresponding fragment f_i . Upon receiving its fragment, P_i will announce to all other parties that it is holding its fragment. Each party will wait for $n - t$ such announcements, but this only guarantees that $n - 2t$ honest parties are holding fragments.

If we could use the higher reconstruction threshold of $n - t$, we could hope to reduce the communication complexity. However, using this higher threshold creates technical difficulties in

³ See, e.g., <https://github.com/AndersTrier/reed-solomon-simd>.

PROTOCOL	COMM/ $(m n)$ (good case) ^a	COMM/ $(m n)$ (worst case) ^b	ROUNDS	BALANCED ^c	CRYPTO
Bracha [Bra87]	$O(n)$	=	3	✓ (SB)	-
ANRX-RBC [ANRX21]	$O(n)$	=	2	✓ (SB)	sig
CT-RBC [CT05]	3	=	3	✓✓ (WB)	hash
ICC2-RBC [CDH ⁺ 22]	3	=	2	✓✓ (WB)	hash+sig
$\mathcal{A}_{\text{bit}}$ [Loc24]	2	=	3	✓✓ (WB)	hash
$\mathcal{A}_{\text{sig}}$ [Loc24]	2	2.5	2	✓✓ (WB)	hash+sig
MiniCast [LS25]	1.5	=	4	✓✓ (WB)	hash
OptMiniCast [LS25]	1.5	2	3	✓✓ (WB)	hash+sig
MiniCast* (§3)	1.5	=	3	✓ (SB)	hash
OptMiniCast* (§4)	1.5	2	2	✓ (SB)	hash+sig
UB-MiniCast (§5)	1.5	=	2	✗ (UB)	hash+sig

(a) and (b) estimate total communication complexity divided by $|m|n$, for large $|m|$, ignoring lower order terms; for optimistic protocols, (a) measures the “good case” where sender does not provably misbehave, while (b) measures the worst case; for other protocols, these are the same, as indicated by =. (c) indicates communication balance: SB = “somewhat balanced” ($\max/\text{avg} = O(1)$), WB = “well balanced” ($\max/\text{avg} \sim 1$), UB = “unbalanced”; for optimistic protocols, this is for the “good case”.

Table 1. Comparison of reliable broadcast protocols

maintaining the completeness property. Locher [Loc24] gives a protocol $\mathcal{A}_{\text{bit}}$ that uses a reconstruction threshold of $n - t$ and achieves communication complexity $\sim 2|m|n$ and round complexity 3. Locher gives a second protocol $\mathcal{A}_{\text{sig}}$ with a round complexity of 2. The communication complexity of $\mathcal{A}_{\text{sig}}$ is $\sim 2|m|n$ unless the sender provably misbehaves, in which case it is $\sim 2.5|m|n$.

Locher and Shoup [LS25] introduce a technique in which, in addition to using a reconstruction threshold of $n - t$, it incorporates a 2-level erasure-coding technique in which each fragment f_i is encoded as a vector of “mini-fragments” $\phi_{i1}, \dots, \phi_{in}$, from any $n - 2t$ of which it is possible to reconstruct f_i . Using mini-fragments, it is possible to ensure that every honest party can eventually obtain its fragment with little communication complexity. Using this fragment/mini-fragment technique, Locher and Shoup give two protocols. As already mentioned, one of these protocols, MiniCast, achieves a round complexity of 4 and a communication complexity of $\sim 1.5|m|n$; moreover, the communication is well balanced. The other protocol, OptMiniCast, achieves a round complexity of 3 and a communication complexity of $\sim 1.5|m|n$, unless the sender *detectably* misbehaves, in which case it is $\sim 2|m|n$.⁴

Table 1 presents a brief summary that compares our new protocols to those mentioned above.

The fragment/mini-fragment technique in [LS25] can be viewed as a variation on the technique of secret sharing using bivariate polynomial interpolation that has been widely used in asynchronous multi-party computation (see [CP17]). Indeed, one could use bivariate polynomial interpolation as an implementation of the fragment/mini-fragment technique, but doing so may not result in the most efficient implementation in practice (the paper [LS25] discussed state-of-the-art erasure-coding software that would not be usable if we insisted on bivariate polynomial interpolation).

Although OptMiniCast has a round complexity of 3, it was shown in [Sho24] how it can be adapted for use in Simplex [CP23], which is a leader-based atomic broadcast protocol, so that in this setting, it effectively has a round complexity of 2. In this way, one obtains an atomic broadcast

⁴ Protocol OptMiniCast does not rely on signatures, but if the sender signs its messages, the protocol achieves the lower communication complexity unless the sender *provably* misbehaves.

protocol that enjoys the same low latency as Simplex, but with much lower and well-balanced communication complexity, which leads to significantly better performance when bandwidth is limited.

Subsequent to [LS25], Alhaddad, Reyzin, and Varia [ARV24] apply the same fragment/mini-fragment technique to the problem of AVID (asynchronous verifiable information dispersal). An AVID protocol has two distinct stages: *dispersal* and *retrieval*. In the dispersal stage, the sender S ensures that its message m is disseminated among the parties $P_1, \dots, P_n$ in such a way that, later, some other party R can reliably retrieve m . It is not required that S or R are among the parties $P_1, \dots, P_n$ or that each party P_i stores the complete message m . Of course, one can always implement reliable broadcast using AVID, by just running the dispersal stage, and then having each party P_i retrieve the message running a separate instance of the retrieval stage.

The main goal of [ARV24] was to reduce the *storage complexity* of AVID protocols, and was not focused per se on the communication or round complexity of reliable broadcast protocols. While [ARV24] makes use of the same fragment/mini-fragment technique introduced in [LS25], they employ the technique in a slightly different way. In [LS25], the sender disperses fragments to all parties, and mini-fragments are only generated by parties holding the message so as to allow other parties to recover their missing fragments as necessary. By contrast, in [ARV24], the sender disperses mini-fragments directly, and the other parties reconstruct their fragments from these mini-fragments.

To be fair, our protocol MiniCast* can essentially be seen as an implementation (as discussed above) of reliable broadcast using the AVID protocol in [ARV24], but with an optimization in which the retrieval stage piggybacks on the last flow of the dispersal stage of the AVID protocol (to get the stated round complexity). While this might be seen as a rather minor observation, we still believe it is worthwhile to present MiniCast* (and OptMiniCast*) in order to paint a more complete picture of the reliable broadcast landscape. For MiniCast* (and OptMiniCast*), the fact that the sender is directly dispersing mini-fragments instead of fragments is what leads to the modest imbalance in communication, as the mini-fragment encoding is inherently more redundant. By contrast, the protocol UB-MiniCast has a very different structure than MiniCast* (and OptMiniCast*).

1.2 Open questions

We raise two open questions. First, can we generalize the lower bound of [Loc24] so that it also applies to protocols that are not necessarily balanced in any way? Even a lower bound of this type that applies only to protocols with round complexity 2 would be interesting. Second, is there a reliable broadcast protocol with round complexity 3, where each party, including the sender, transmits $\sim 1.5|m|$ bits as $|m| \rightarrow \infty$?

1.3 Formal definitions

We refer the reader to Section 2 of [LS25] for more formal definitions of the communication and security model.

2 Data encoding logic

We use the same data encoding logic introduced in [LS25], but with a few changes. We review this logic and describe these changes here.

2.1 Erasure codes

For integer parameters $k \geq d \geq 1$, a (k, d) -**erasure code** encodes a bit string m as a vector of k **fragments**, $f_1, \dots, f_k$, in such a way that any d such fragments may be used to efficiently reconstruct m . Note that for variable-length m , the reconstruction algorithm also takes as input the length ℓ of m . The reconstruction algorithm may fail (for example, a formatting error)—if it fails it returns $\perp$, while if it succeeds it returns a message that when re-encoded will yield k fragments that agree with the original subset of d fragments. We assume that all fragments have the same size, which is determined as a function of k , d , and ℓ . As in [LS25], we use the upper bound

$$\ell/d + O(\log(k))$$

on fragment size.

As in [LS25], our reliable broadcast protocol uses two erasure codes.

- *An $(n, n - t)$ -erasure code.*

This is used to encode the message m to be broadcast by our protocol as a vector of n fragments, $f_1, \dots, f_n$, any $n - t$ of which may be used to efficiently reconstruct the message m . If $|m| = \ell$, then the size of each fragment f_i , which is a function of ℓ , n , and t , is at most

$$\frac{\ell}{n - t} + O(\log(n)) \leq \frac{3}{2n}\ell + O(\log(n)).$$

- *An $(n, n - 2t)$ -erasure code.*

This is used to encode a fragment f_i as above as a vector of n “mini-fragments” $\phi_{i1}, \dots, \phi_{in}$, any $n - 2t$ of which may be used to reconstruct the fragment f_i . If $|m| = \ell$, then the size of each mini-fragment ϕ_{ij} , which is a function of ℓ , n , and t , is at most

$$\frac{\ell}{(n - t)(n - 2t)} + O(\log(n)) \leq \frac{9}{2n^2}\ell + O(\log(n)).$$

2.2 Merkle trees

As in [LS25], our reliable broadcast protocol will also make use of Merkle trees. Recall that a Merkle tree allows one party P to commit to a vector of values $(v_1, \dots, v_k)$ using a collision-resistant hash function by building a (full) binary tree whose leaves are the hashes of $v_1, \dots, v_k$, and where each internal node of the tree is the hash of its two children. The root r of the tree is the commitment. Party P may “open” the commitment at a position $i \in [k]$ by revealing v_i along with a “validation path” π_i , which consists of the siblings of all nodes along the path in the tree from the hash of v_i to the root r . We call π_i a **validation path for v_i under r at position i** . Such a validation path is checked by recomputing the nodes along the corresponding path in the tree, and verifying that the recomputed root is equal to the given commitment r . The collision resistance of the hash function ensures that P cannot open the commitment to two different values at a given position.

As in [LS25], we shall actually make use of a two-level Merkle tree structure. For a given message m , the top level Merkle tree will be rooted at r and have leaves $r_1, \dots, r_n$, where each r_i is itself the root of a Merkle tree whose leaves are the hashes of mini-fragments $\phi_{i1}, \dots, \phi_{in}$ of the fragment f_i of m . We will therefore have validation paths π_i for r_i under r at position i , as well as validation paths π_{ij} for ϕ_{ij} under r_i at position j .

Each validation path π_i and π_{ij} will have size $O(\kappa \log(n))$, where κ is the output length of the hash function.

2.3 Encoding and decoding functions

We define *tags*, *certified fragments*, and *certified mini-fragments* as in [LS25]. We repeat these definitions here.

Tags. For a given message length ℓ and Merkle tree root r , we define the **tag** $\tau := (\ell, r)$.

Certified fragments. For a tag $\tau = (\ell, r)$, we shall call (f_i, π_i) a **certified fragment for τ at position i** if

- f_i has the correct length of a fragment for a message of length ℓ , and
- when we compute the mini-fragments $\phi_{i1}, \dots, \phi_{in}$ for f_i and build the Merkle tree rooted at r_i whose leaves are the hashes of $\phi_{i1}, \dots, \phi_{in}$, we find that π_i is a correct validation path for r_i under r at position i .

Certified mini-fragments. For a tag $\tau = (\ell, r)$, we shall call $(\phi_{ij}, \pi_{ij}, \pi_i)$ a **certified mini-fragment for τ at position (i, j)** if

- ϕ_{ij} has the correct length of a mini-fragment for a message of length ℓ ,
- π_{ij} is a validation path for ϕ_{ij} under a root r_i , and
- π_i is a correct validation path for r_i under r .

We define several functions that will be used in our protocol. All of these functions take n and t as implicit parameters.

The Encode function. We define the *Encode* function slightly differently than in [LS25] — the only difference being that we include more items in the output, which will be used by our protocols, and which anyway are computed along the way. The *Encode* function takes as input a message m . It builds a two-level Merkle tree for m as above with root r (encoding m as a vector of fragments, encoding each fragment as a vector of mini-fragments, and then building a two-level Merkle tree whose leaves are the hashes of all of these mini-fragments). It returns

$$\left(\tau, \{(f_i, \pi_i)\}_{i \in [n]}, \{(\phi_{ij}, \pi_{ij})\}_{i, j \in [n]} \right),$$

where τ is the tag (ℓ, r) , ℓ is the length of m , each (f_i, π_i) is a certified fragment for τ at position i , and each $(\phi_{ij}, \pi_{ij}, \pi_i)$ is a certified mini-fragment at position (i, j) .

The Decode function. We define the *Decode* function slightly differently than in [LS25]. The *Decode* function takes as input

$$\left(\tau, \{ \{ (f_j, \pi_j) \}_{j \in \mathcal{J}} \} \right),$$

where $\tau = (\ell, r)$ is a tag, $\mathcal{J}$ is a subset of $[n]$ of size $n - t$, and each (f_j, π_j) is a certified fragment for τ at position j . It first reconstructs a message m' from the fragments $\{f_j\}_{j \in \mathcal{J}}$, using the size parameter ℓ . If $m' = \perp$, it returns $\perp$. Otherwise, it encodes m' as a vector of fragments, encodes each of these fragments as a vector of mini-fragments, and builds a two-level Merkle tree with root r' from the mini-fragments. If $r' \neq r$, it returns $\perp$. Otherwise, it returns m' .

The MiniDecode function. We need a function, similar to *Decode*, that decodes sets of mini-fragments. This function, called *MiniDecode*, is analogous to the *Recover* function in [LS25], but differs in that it performs a consistency check on the mini-fragments. Function *MiniDecode* takes as input

$$(i, \tau, \pi_i, \{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}}),$$

where $i \in [n]$, $\tau = (\ell, r)$ is a tag, $\mathcal{J}$ is a subset of $[n]$ of size $n - 2t$, and each $(\phi_{ij}, \pi_{ij}, \pi_i)$ is a certified mini-fragment for τ at position (i, j) . It first reconstructs a fragment f'_i from the mini-fragments $\{\phi_{ij}\}_{j \in \mathcal{J}}$ using size parameter ℓ . If $f'_i = \perp$, it returns $\perp$. Otherwise, it checks if (f'_i, π_i) is a certified fragment for τ at position i . If so, it outputs f'_i , and otherwise, it outputs $\perp$.

2.4 The basic data flow

In our protocol, the sender will take a message m and compute

$$(\tau, \{(f_i, \pi_i)\}_{i \in [n]}, \{(\phi_{ij}, \pi_{ij})\}_{i,j \in [n]}) \leftarrow \text{Encode}(m)$$

and send to each party P_i the tag τ along with the collection of certified mini-fragments

$$\{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n.$$

Each such P_i will echo these certified mini-fragments, sending $(\phi_{ji}, \pi_{ji}, \pi_j)$ to each party P_j . This will contribute at most

$$9\ell + O(\kappa n^2 \log(n)) \tag{1}$$

to the communication complexity.

By obtaining a collection $\{(\phi_{ij}, \pi_{ij}, \pi_i)\}_{j \in \mathcal{J}}$ of $n - 2t$ certified mini-fragments for the tag τ , party P_i can then compute

$$f_i \leftarrow \text{MiniDecode}(i, \tau, \pi_i, \{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}})$$

Assuming $f_i \neq \perp$, party P_i then broadcasts the certified fragment (f_i, π_i) to each other party. This contributes at most

$$1.5\ell n + O(\kappa n^2 \log(n)) \tag{2}$$

to the communication complexity.

By obtaining a collection $\{(f_j, \pi_j)\}_{j \in \mathcal{J}}$ of $n - t$ certified fragments for the tag τ , party P_i can then compute

$$m \leftarrow \text{Decode}(i, \tau, \{(f_j, \pi_j)\}_{j \in \mathcal{J}}).$$

Assuming $m \neq \perp$, party P_i then outputs m .

Combining (1) and (2), the total communication complexity is at most

$$1.5\ell n + 9\ell + O(\kappa n^2 \log(n)).$$

3 The main protocol

Our new reliable broadcast protocol, which we call Π_{MiniCast^*} , is presented in Fig. 1. We express the logic for the sender as a separate process, even though the sending party is also one of the receiving parties. For each receiving party, the main logic is presented as a loop, and in each loop iteration the party waits for one of several conditions to hold, triggering a reaction. These conditions are based on local state plus an implicit pool of received messages.

This protocol has the same basic structure as Bracha’s original broadcast protocol [Bra87], with three rounds, which we call *disperse*, *echo*, and *vote*. The main difference between this and Bracha’s protocol is that here, data is disseminated as outlined in Section 2.4. At a high level, the protocol proceeds as follows:

1. In the disperse round, the sender sends to each party P_i the tag τ along with the collection of certified mini-fragments $\{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$.
2. In the echo round, each party P_i “echoes τ ” by broadcasting the message (echo, τ) , and sends to each party P_j the certified mini-fragment $(\phi_{ji}, \pi_{ji}, \pi_j)$.
3. In the vote round, each P_i will “vote for τ ” by broadcasting the message (vote, τ) . It does so only after two conditions hold:
 - (a) it has received either $n - t$ echoes of τ or $t + 1$ votes for τ , and
 - (b) it has received a set of $n - 2t$ certified mini-fragments $(\phi_{ij}, \pi_{ij}, \pi_i)$ for τ , which allows it to reconstruct its fragment f_i for τ .
 In addition to voting for τ , it also broadcasts its certified fragment (f_i, π_i) .
4. Upon receiving $n - t$ votes for τ , as well as $n - t$ certified fragments for τ , each party reconstructs a message m from these fragments and then outputs m .

Theorem 3.1. *Assuming collision resistance of the underlying hash functions, Protocol Π_{MiniCast^*} satisfies the correctness and completeness property (with overwhelming probability, for any polynomial-time adversary).*

Proof. We first deal with the case where the sender is corrupt. In this case, we need to prove the following: *if any honest party outputs a message, then all honest parties eventually output the same message.*

In proving this, we first recall a basic property of Bracha’s protocol, which applies equally well here: if any honest party votes for a tag τ , then

- (i) all honest parties vote for the same tag τ , and
- (ii) at least $n - t - t'$ honest parties must have echoed τ , where t' is the number of corrupt parties.

Note that (i) follows directly from (ii).

Consider the first point in time where some honest party P outputs a message m . Since that party received $n - t$ votes for τ , it must be the case that $n - 2t \geq t + 1$ honest parties already voted for τ . Therefore, all honest parties will eventually receive $t + 1$ votes for some tag, and by property (i) above, this tag must be τ . Moreover, by property (ii) above, each honest party will eventually receive the $n - 2t$ certified mini-fragments for τ it needs to obtain its fragment for τ . Therefore, each honest party P_i will eventually vote for τ — collision resistance ensures that when P_i executes *MiniDecode*, it will recover the same fragment f_i that P constructed when it performed the consistency check in its execution of *Decode*. Therefore, each honest party will eventually receive

Π_{MiniCast^*}

```

// Sender  $S$  with input  $m$ 
(  $\tau, \{(f_i, \pi_i)\}_{i \in [n]}, \{(\phi_{ij}, \pi_{ij})\}_{i,j \in [n]} \} \leftarrow \text{Encode}(m)$ 
for  $i \in [n]$ : send (disperse,  $\tau, \{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$ ) to  $P_i$ 

// Receiving party  $P_i$ 
 $\text{echoed} \leftarrow \text{voted} \leftarrow \text{delivered} \leftarrow \text{false}$ 

repeat forever: wait until either:
  not  $\text{echoed}$  and for some  $\tau, \{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$ : received (disperse,  $\tau, \{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$ ) from  $S \Rightarrow$ 
     $\text{echoed} \leftarrow \text{true}$ 
    if  $(\phi_{ji}, \pi_{ji}, \pi_j)$  is a certified mini-fragment for  $\tau$  at position  $(j, i)$  for each  $j \in [n]$  then
      send (echo,  $\tau$ ) to  $P_1, \dots, P_n$ 
      for  $j \in [n]$ : send (mini-frag,  $\tau, (\phi_{ji}, \pi_{ji}, \pi_j)$ ) to  $P_j$ 

  not  $\text{voted}$  and for some  $\tau, \pi_i, \mathcal{J} \subseteq [n]$  of size  $n - 2t$ , and  $\{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}}$ :
    ( received (echo,  $\tau$ ) from  $n - t$  distinct parties or (vote,  $\tau$ ) from  $t + 1$  distinct parties )
    and for all  $j \in \mathcal{J}$ : received (mini-frag,  $\tau, (\phi_{ij}, \pi_{ij}, \pi_i)$ ) from  $P_j$ 
    and  $(\phi_{ij}, \pi_{ij}, \pi_i)$  is a certified mini-fragment for  $\tau$  at position  $(i, j) \Rightarrow$ 
       $\text{voted} \leftarrow \text{true}$ 
       $f_i \leftarrow \text{MiniDecode}(i, \tau, \pi_i, \{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}})$ 
      if  $f_i \neq \perp$  then
        send (vote,  $\tau$ ) to  $P_1, \dots, P_n$ 
        send (frag,  $\tau, (f_i, \pi_i)$ ) to  $P_1, \dots, P_n$ 

  not  $\text{delivered}$  and for some  $\tau, \mathcal{J} \subseteq [n]$  of size  $n - t$ , and  $\{(f_j, \pi_j)\}_{j \in \mathcal{J}}$ :
    received (vote,  $\tau$ ) from  $n - t$  distinct parties
    and for all  $j \in \mathcal{J}$ : received (frag,  $\tau, f_j, \pi_j$ ) from  $P_j$ 
    and  $(f_j, \pi_j)$  is a certified fragment for  $\tau$  at position  $j \Rightarrow$ 
       $\text{delivered} \leftarrow \text{true}$ 
       $m \leftarrow \text{Decode}(\tau, \{(f_j, \pi_j)\}_{j \in \mathcal{J}})$ 
      if  $m \neq \perp$  then output  $m$ 

```

Fig. 1. The MiniCast* reliable broadcast protocol

$n - t$ votes for τ , as well as $n - t$ certified fragments for τ , and collision resistance ensures that it will reconstruct and output m .

That completes the proof for the case where the sender is corrupt. Now consider the case where the sender is honest and sends a message m . In this case, we need to prove the following: *every honest party will eventually output m .*

To prove this, suppose the tag for m is τ . Then it follows that every honest party will eventually echo τ , which implies that every honest party will eventually vote for τ , which implies every honest party will eventually receive $n - t$ votes and fragments for τ , and collision resistance implies every honest party will eventually output m . $\square$

3.1 Message and communication complexity

The message complexity of Π_{MiniCast^*} is obviously $O(n^2)$.

As sketched in Section 2, the communication complexity of Π_{MiniCast^*} is at most

$$1.5\ell n + 9\ell + O(\kappa n^2 \log(n)), \quad (3)$$

where κ is the output length of the hash function. Here, ℓ represents a bound on the message length. When the sender S is honest, we can take ℓ to be the length of the message m that S is sending. When S is corrupt, we can take ℓ to be the value that is in the dispersed tag τ —honest parties will not transmit any data until the tag is agreed upon, and so this value is well defined, and the size of the fragments and mini-fragments transmitted by the honest parties will be consistent with a message of length ℓ (and if any honest party ultimately delivers a message, it will be of length ℓ). Any particular application may impose a bound on the message length and the honest parties can enforce this bound when they execute the protocol.

We also note that the communication is somewhat balanced among all parties. Each party transmits (and receives) $1.5\ell + O(\ell/n + \kappa n \log(n))$ bits of data, except for the sender, who transmits an additional $4.5\ell + O(\kappa n^2 \log(n))$ bits of data. In fact, using the same “balancing technique” from Section 4.3 of [LS25] (or Section 4.1.2 of the full version), the sender only needs to send this much data and nothing more.

4 An optimistic 2-round variant

We now present an “optimistic” version of our protocol Π_{MiniCast^*} that has better round complexity—just 2 rounds in the “good case” where the sender is honest, instead of 3. This protocol is based on the reliable broadcast protocol in [ANRX21], which makes use of threshold signatures. When the sender is honest, its communication complexity is $\sim 1.5|m|n$ as $|m| \rightarrow \infty$, but this may increase to $\sim 2|m|n$ if the sender provably misbehaves.

The protocol makes use of an $(n - t)$ -out-of- n threshold signature scheme. For such a scheme, we refer to a *signature share* and a *signature certificate*: signature shares from $n - t$ distinct parties on a given message may be combined to form a signature certificate on that message. This can be implemented as just a set of ordinary signatures, or as an aggregate signature scheme (such as one based on BLS signatures [BLS01] as in [BDN18]) or as a threshold version of an ordinary signature scheme (such as one again based on BLS signatures as in [Bol03]). The second and third implementations will result in much more compact threshold signatures. The third implementation requires a set-up phase to distribute shares of a signing key.

The security property for such a threshold signature scheme may be stated as follows.

Quorum Size Property: It is infeasible to produce a signature certificate on a message m , unless $n - t - t'$ honest parties have issued signature shares on m , where $t' \leq t$ is the number of corrupt parties.

Under our assumption that the number of corrupt parties is strictly less than $n/3$, one can easily establish the following standard property.

Quorum Intersection Property: It is infeasible to produce signature certificates on two distinct messages m and m' , unless at least one honest party issued signature shares on both m and m' .

Our optimistic reliable broadcast protocol, which we call $\Pi_{\text{OptMiniCast}^*}$, is presented in Fig. 2. The protocol collapses the round structure of Π_{MiniCast^*} to just two rounds: *disperse* and *echo*. The main ideas of the protocol are as follows:

1. In the *disperse* round, the sender includes the fragment f_i in the message it sends to P_i , along with all the other information sent in Π_{MiniCast^*} .
2. In the *echo* round, in addition to all the other information sent in Π_{MiniCast^*} , each party P_i broadcasts its fragment f_i as well as a signature share σ_i on a message consisting of the tag τ as well as a protocol instance ID. For brevity, in the protocol description, we refer to this simply as a signature share on τ (and similarly, a signature certificate on τ is really a signature certificate on a message consisting of τ and a protocol instance ID). Note that broadcasting f_i at this stage is rather optimistic, as we have not yet reached agreement on a tag, and so this may well be the “wrong” fragment.
3. If a party receives $n - t$ signature shares or a signature certificate on a tag τ^* , then τ^* is considered *certified*. In either case, the party broadcasts a signature certificate on τ^* (computing a signature certificate from signature shares, if necessary). By the Quorum Intersection Property, all honest parties will agree on the certified tag τ^* . Moreover, by the Quorum Size Property, if any honest party sees a certified tag τ^* , we can be sure that at least $n - 2t$ honest parties have sent out corresponding mini-fragments to all parties, which ensures that all honest parties can eventually recover their own fragment for τ^* even if they do not receive one from the sender.
4. If a party P_i obtains its fragment f_i for τ^* through the above recovery mechanism, it will broadcast f_i . Of course, P_i takes care not to broadcast the same fragment twice. However, it could broadcast two fragments in total—a fragment for the “wrong” tag in the *echo* round and a fragment for the “right” tag in the recovery mechanism. This can only happen if the sender “equivocates”—sending different tags to different honest parties in the *disperse* round.

Theorem 4.1. *Assuming collision resistance of the underlying hash functions, and the security of the underlying threshold signature scheme, Protocol $\Pi_{\text{OptMiniCast}^*}$ satisfies the correctness and completeness property (with overwhelming probability, for any polynomial-time adversary).*

In addition, if the sender is honest, then no honest party broadcasts more than a single fragment; otherwise, the number of honest parties that send two fragments is at most t .

Proof. We first deal with the case where the sender is corrupt. In this case, we need to prove the following: *if any honest party outputs a message, then all honest parties eventually output the same message.*

Consider the first point in time where some honest party P outputs a message m with certified tag τ^* . By the Quorum Intersection Property and the logic of the protocol, every honest party will eventually certify the same tag τ^* . Moreover, by the Quorum Size Property, each honest party will

```

// Sender S with input m
(  $\tau, \{(f_i, \pi_i)\}_{i \in [n]}, \{(\phi_{ij}, \pi_{ij})\}_{i,j \in [n]} \} \leftarrow \text{Encode}(m)$ 
for  $i \in [n]$  : send (disperse,  $\tau, f_i, \{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$ ) to  $P_i$ 

// Receiving party  $P_i$ 
echoed  $\leftarrow$  recovered  $\leftarrow$  delivered  $\leftarrow$  false,  $\tau_{\text{echo}} \leftarrow \tau^* \leftarrow \perp$ 

repeat forever: wait until either:
  not (echoed or recovered) and for some  $\tau, f_i, \{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$ :
    received (disperse,  $\tau, f_i, \{(\phi_{ji}, \pi_{ji}, \pi_j)\}_{j=1}^n$ ) from  $S \Rightarrow$ 
      echoed  $\leftarrow$  true
      if  $(f_i, \pi_i)$  is a certified fragment for  $\tau$  at position  $i$ 
        and  $(\phi_{ji}, \pi_{ji}, \pi_j)$  is a certified mini-fragment for  $\tau$  at position  $(j, i)$  for each  $j \in [n]$  then
          compute  $P_i$ 's signature share  $\sigma_i$  on  $\tau$ 
          send (echo,  $\tau, \sigma_i$ ) to  $P_1, \dots, P_n$ 
          for  $j \in [n]$ : send (mini-frag,  $\tau, (\phi_{ji}, \pi_{ji}, \pi_j)$ ) to  $P_j$ 
          send (frag,  $\tau, (f_i, \pi_i)$ ) to  $P_1, \dots, P_n$ 
           $\tau_{\text{echo}} \leftarrow \tau$ 

   $\tau^* = \perp$  and for some  $\tau, \mathcal{J} \subseteq [n]$  of size  $n - t$ , and  $\{\sigma_j\}_{j \in \mathcal{J}}$ :
    for all  $j \in \mathcal{J}$ : received (echo,  $\tau, \sigma_j$ ) from  $P_j$ 
      and  $\sigma_j$  is a valid signature share from  $P_j$  on  $\tau \Rightarrow$ 
         $\tau^* \leftarrow \tau$ 
        compute a signature certificate for  $\tau$  from  $\{\sigma_j\}_{j \in \mathcal{J}}$ 
        send (cert,  $\tau, \sigma$ ) to  $P_1, \dots, P_n$ 

   $\tau^* = \perp$  and for some  $\tau, \sigma$ :
    received (cert,  $\tau, \sigma$ ) and  $\sigma$  is a valid signature certificate on  $\tau \Rightarrow$ 
       $\tau^* \leftarrow \tau$ 
      send (cert,  $\tau, \sigma$ ) to  $P_1, \dots, P_n$ 

  not recovered and  $\tau^* \notin \{\perp, \tau_{\text{echo}}\}$  and for some  $\pi_i, \mathcal{J} \subseteq [n]$  of size  $n - 2t$ , and  $\{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}}$ :
    for all  $j \in \mathcal{J}$ : received (mini-frag,  $\tau^*, (\phi_{ij}, \pi_{ij}, \pi_i)$ ) from  $P_j$ 
      and  $(\phi_{ij}, \pi_{ij}, \pi_i)$  is a certified mini-fragment for  $\tau^*$  at position  $(i, j) \Rightarrow$ 
        recovered  $\leftarrow$  true
         $f_i \leftarrow \text{MiniDecode}(i, \tau^*, \pi_i, \{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}})$ 
        if  $f_i \neq \perp$  then send (frag,  $\tau^*, (f_i, \pi_i)$ ) to  $P_1, \dots, P_n$ 

  not delivered and  $\tau^* \neq \perp$  and for some  $\mathcal{J} \subseteq [n]$  of size  $n - t$  and  $\{(f_j, \pi_j)\}_{j \in \mathcal{J}}$ :
    for all  $j \in \mathcal{J}$ : received (frag,  $\tau^*, (f_j, \pi_j)$ ) from  $P_j$ 
      and  $(f_j, \pi_j)$  is a certified fragment for  $\tau^*$  at position  $j \Rightarrow$ 
        delivered  $\leftarrow$  true
         $m \leftarrow \text{Decode}(\tau^*, \{(f_j, \pi_j)\}_{j \in \mathcal{J}})$ 
        if  $m \neq \perp$  then output  $m$ 

```

Fig. 2. The OptMiniCast* reliable broadcast protocol

eventually receive the $n - 2t$ certified mini-fragments for τ^* it needs to obtain and broadcast its fragment for τ^* (if it did not already receive it directly from the sender). Therefore, each honest party will eventually receive $n - t$ certified fragments for τ^* and then reconstruct and output m .

Note that some honest parties may broadcast two fragments: one when they echo some tag $\tau \neq \tau^*$, and then again when they broadcast a fragment for τ^* in the recovery mechanism. Now, by the Quorum Size Property, at least $n - t - t'$ out of the $n - t'$ honest parties must have echoed $\tau = \tau^*$, and therefore will only broadcast a single fragment. So at most $(n - t') - (n - t - t') = t$ will broadcast two fragments.

Now consider the case where the sender is honest and sends a message m . In this case, we need to prove the following: *every honest party will eventually output m .*

To prove this, suppose the tag for m is τ . Then it follows that every honest party will eventually echo τ , which implies that every honest party will eventually certify τ and also reconstruct and broadcast its fragment for τ . Therefore, each honest party will eventually receive $n - t$ certified mini-fragments for τ and then reconstruct and output m .

It is also clear that in this case, every honest party will only broadcast a fragment for τ . $\square$

4.1 Message and communication complexity

The message complexity of $\Pi_{\text{OptMiniCast}^*}$ is still $O(n^2)$.

If the sender is honest, then by [Theorem 4.1](#), no honest parties broadcast more than one fragment, and it follows that the communication complexity of $\Pi_{\text{OptMiniCast}^*}$ in this case is at most

$$1.5\ell n + 10.5\ell + O(\kappa n^2 \log(n) + \lambda n^2),$$

where ℓ is the length of the message m that S is sending, and λ is a bound on the length of a signature share or certificate. The additional 1.5ℓ compared to Π_{MiniCast^*} is due to the n fragments of size $1.5\ell/n$ sent by the sender in the *disperse* round.

We also note that in this case, the communication is somewhat balanced among all parties. Each party transmits (and receives) $1.5\ell + O(\ell/n + \kappa n \log(n) + \lambda n)$ bits of data, except for the sender, who transmits an additional $6\ell + O(\kappa n^2 \log(n))$ bits of data. In fact, using the same “balancing technique” from Section 4.3 of [\[LS25\]](#) (or Section 4.1.2 of the full version), the sender only needs to send this much data plus $O(\lambda n)$ bits.

If the sender is corrupt, then by [Theorem 4.1](#), at most $t < n/3$ honest parties broadcast two fragments. It follows that the communication complexity of $\Pi_{\text{OptMiniCast}^*}$ in this case is at most

$$2\ell n + 10.5\ell + O(\kappa n^2 \log(n) + \lambda n^2).$$

Here, ℓ represents a bound on the message length. Unlike in [Section 3.1](#), honest parties may transmit fragments before a tag is agreed upon; therefore, we take ℓ to be some application-dependent bound on message length. As mentioned before, such a bound may be enforced by honest parties.

As noted above, an honest party will broadcast two fragments only if the sender “equivocates”—sending different tags to different honest parties in the *disperse* round. One can limit the impact of this in two ways:

1. The party that broadcasts two fragments will detect that the sender is corrupt, and can ignore all messages from that sender in future executions of the protocol.

2. If we modify the protocol so that the sender must include a signature on the tag (and protocol instance ID) in the *disperse* round, and this signature is passed along in the *echo* round, then any such equivocation will eventually result in all honest parties obtaining an “equivocation proof”, consisting of signatures on conflicting tags. Such an equivocation proof may be used to take punitive action against the sender, which presumably makes any such equivocation unlikely to happen in the first place.

5 An unbalanced 2-round variant

The protocols we have presented so far are somewhat balanced. In this section, we consider a completely unbalanced protocol, in which most of the data is transmitted by the sender. As already mentioned in the introduction, in symmetric applications where all parties simultaneously broadcast in each stage of the protocol, such imbalance has no practical impact. Like protocol $\Pi_{\text{OptMiniCast}^*}$, the protocol we present here relies on threshold signatures. It also has a round complexity of just 2 in the “good case” where the sender is honest. However, unlike $\Pi_{\text{OptMiniCast}^*}$, its communication complexity *in the worst case* is just $\sim 1.5|n|$ as $|m| \rightarrow \infty$. That said, we do not necessarily recommend it over $\Pi_{\text{OptMiniCast}^*}$ in all situations, for reasons we discuss below.

Like protocol $\Pi_{\text{OptMiniCast}^*}$, we need an $(n - t)$ -out-of n threshold signature scheme. However, to achieve the desired worst-case bound on communication complexity, we need an *accountable* threshold signature scheme [BPR24]. Roughly speaking, this means that the set of signatories can be computed from a signature. We write this computation as $\mathcal{J} \leftarrow \text{Signatories}(\sigma)$. An essential security property is that if σ is a valid signature on a message and $j \in \mathcal{J}$ is the index of an honest party P_j , then P_j must have issued a signature share on the signed message.

We can implement accountable threshold signatures as either a set of signatures or as an aggregate signature scheme. Such implementations may lead to signature certificates of size linear in n . For the case of BLS-based aggregate signatures, we need just n *bits* to identify the set of signatories.

Our unbalanced 2-round reliable broadcast protocol, which we call $\Pi_{\text{UB-MiniCast}}$, is presented in Fig. 3. The main ideas of the protocol are as follows.

1. In the *disperse* round, given a message m , the sender sends each party the message $(\text{disperse}, m)$.
2. In the *echo* round, upon receiving $(\text{disperse}, m)$ from the sender, each party P_i computes

$$(\tau, \{(f_i, \pi_i)\}_{i \in [n]}, \{(\phi_{ij}, \pi_{ij})\}_{i,j \in [n]}) \leftarrow \text{Encode}(m).$$

It then computes its signature share σ_i on τ , sends the message $(\text{echo}, \tau, \sigma_i)$ to all parties, and sends the message $(\text{mini-frag}, \tau, (\phi_{ji}, \pi_{ji}, \pi_j))$ to each party P_j .

3. Just as in $\Pi_{\text{OptMiniCast}^*}$, if a party receives $n - t$ signature shares or a signature certificate on a tag τ^* , then τ^* is considered *certified*, and the party broadcasts a signature certificate on τ^* . In addition, it records the set $\mathcal{J}^*$ of signatories on the certificate. Again, just as in $\Pi_{\text{OptMiniCast}}$, we can be sure that all honest parties can eventually recover their own fragment for τ^* . When a party does recover its own fragment, it will send it to all parties P_j for $j \in [n] \setminus \mathcal{J}^*$.
4. As soon as τ^* is certified by a party, and that party has obtained the corresponding message m , it can output the message. It can obtain m in one of two ways: either directly from the sender or via the recovery mechanism. When the sender is honest, every party can output m immediately after the *disperse* and *echo* rounds.


```

// Sender S with input m
send (disperse, m) to  $P_1, \dots, P_n$ 

// Receiving party  $P_i$ 
echoed  $\leftarrow$  recovered  $\leftarrow$  delivered  $\leftarrow$  false,  $\tau_{\text{echo}} \leftarrow \tau^* \leftarrow \perp$ 

repeat forever: wait until either:
  not echoed and for some  $m$ : received (disperse,  $m$ ) from  $S \Rightarrow$ 
    echoed  $\leftarrow$  true
    (a)  $(\tau, \{(f_i, \pi_i)\}_{i \in [n]}, \{(\phi_{ij}, \pi_{ij})\}_{i,j \in [n]}) \leftarrow \text{Encode}(m)$ 
        compute  $P_i$ 's signature share  $\sigma_i$  on  $\tau$ 
        send (echo,  $\tau, \sigma_i$ ) to  $P_1, \dots, P_n$ 
        for  $j \in [n]$ : send (mini-frag,  $\tau, (\phi_{ji}, \pi_{ji}, \pi_j)$ ) to  $P_j$ 
         $\tau_{\text{echo}} \leftarrow \tau, m_{\text{echo}} \leftarrow m$ 

     $\tau^* = \perp$  and for some  $\tau, \mathcal{J} \subseteq [n]$  of size  $n - t$ , and  $\{\sigma_j\}_{j \in \mathcal{J}}$ :
      for all  $j \in \mathcal{J}$ : received (echo,  $\tau, \sigma_j$ ) from  $P_j$ 
      and  $\sigma_j$  is a valid signature share from  $P_j$  on  $\tau \Rightarrow$ 
         $\tau^* \leftarrow \tau, \mathcal{J}^* \leftarrow \mathcal{J}$ 
        compute a signature certificate for  $\tau$  from  $\{\sigma_j\}_{j \in \mathcal{J}}$ 
        send (cert,  $\tau, \sigma$ ) to  $P_1, \dots, P_n$ 

     $\tau^* = \perp$  and for some  $\tau, \sigma$ :
      received (cert,  $\tau, \sigma$ ) and  $\sigma$  is a valid signature certificate on  $\tau \Rightarrow$ 
         $\tau^* \leftarrow \tau, \mathcal{J}^* \leftarrow \text{Signatories}(\sigma)$ 
        send (cert,  $\tau, \sigma$ ) to  $P_1, \dots, P_n$ 

  not recovered and  $\tau^* \neq \perp$  and for some  $\pi_i, \mathcal{J} \subseteq [n]$  of size  $n - 2t$ , and  $\{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}}$ :
    for all  $j \in \mathcal{J}$ : received (mini-frag,  $\tau^*, (\phi_{ij}, \pi_{ij}, \pi_i)$ ) from  $P_j$ 
    and  $(\phi_{ij}, \pi_{ij}, \pi_i)$  is a certified mini-fragment for  $\tau^*$  at position  $(i, j) \Rightarrow$ 
      recovered  $\leftarrow$  true
      (b)  $f_i \leftarrow \text{MiniDecode}(i, \tau^*, \pi_i, \{(\phi_{ij}, \pi_{ij})\}_{j \in \mathcal{J}})$  // note:  $f_i \neq \perp$ 
      (c) for  $j \in [n] \setminus \mathcal{J}^*$ : send (frag,  $\tau^*, (f_i, \pi_i)$ ) to  $P_j$ 

  not delivered and  $\tau^* \neq \perp$  and  $\tau^* = \tau_{\text{echo}} \Rightarrow$ 
    delivered  $\leftarrow$  true, output  $m_{\text{echo}}$ 

  not delivered and  $\tau^* \neq \perp$  and for some  $\mathcal{J} \subseteq [n]$  of size  $n - t$  and  $\{(f_j, \pi_j)\}_{j \in \mathcal{J}}$ :
    for all  $j \in \mathcal{J}$ : received (frag,  $\tau^*, (f_j, \pi_j)$ ) from  $P_j$ 
    and  $(f_j, \pi_j)$  is a certified fragment for  $\tau^*$  at position  $j \Rightarrow$ 
    (d) delivered  $\leftarrow$  true,  $m \leftarrow \text{Decode}(\tau^*, \{(f_j, \pi_j)\}_{j \in \mathcal{J}})$ , output  $m$  // note:  $m \neq \perp$ 

```

Fig. 3. The UB-MiniCast reliable broadcast protocol

The following notes are worth mentioning:

1. The decoding steps at the lines marked (b) and (d) are guaranteed to succeed. This is because by this time, we know that at least one honest party has already performed the the *Encode* computation at line (a).
2. Instead of obtaining f_i via decoding at line (b), party P_i will typically be able to obtain it from the *Encode* computation it made at line (a).
3. At line (d), party P_i only sends f_i to parties P_j with $j \in [n] \setminus \mathcal{J}^*$. This is safe, because parties P_j with $j \in \mathcal{J}^*$ already have the message and so do not need f_i . The set $[n] \setminus \mathcal{J}^*$ has size $t < n/3$, which is crucial to get the desired worst-case bound on communication complexity. (This idea was used in a similar setting by [Loc24].) While party P_i sends f_i “proactively”, it is also possible to implement the protocol so that it only sends it “on demand” from parties that request it. This can lead to even lower communication complexity in the common case and reduce pressure on the download bandwidth of parties. We discuss this optimization in more detail below.

Theorem 5.1. *Assuming collision resistance of the underlying hash functions, and the security of the underlying threshold signature scheme (including accountability), Protocol $\Pi_{\text{UB-MiniCast}}$ satisfies the correctness and completeness property (with overwhelming probability, for any polynomial-time adversary).*

We omit the proof of this theorem, as it follows quite easily from the sketch of the main ideas presented above.

5.1 Message and communication complexity

The message complexity of $\Pi_{\text{UB-MiniCast}}$ is still $O(n^2)$.

The communication complexity of $\Pi_{\text{UB-MiniCast}}$ is at most

$$1.5\ell n + 4.5\ell + O(\kappa n^2 \log(n) + \lambda n^2),$$

where ℓ is the length of the message m that S is sending, and λ is a bound on the length of a signature share or certificate. This bound holds even when the sender S is corrupt (in which case ℓ represents a bound on the message length). The main observation to make in establishing this bound is that each party sends its fragment to less than $n/3$ parties, which contributes a total of

$$\sim n \cdot \frac{3}{2} \frac{\ell}{n} \cdot \frac{1}{3} n = \frac{1}{2} \ell n$$

to the communication complexity, in addition to ℓn contributed in the *disperse* round.

5.2 Some practical concerns

As mentioned above, we do not necessarily recommend $\Pi_{\text{UB-MiniCast}}$ over $\Pi_{\text{OptMiniCast}^*}$ in all situations.

A first concern is that $\Pi_{\text{UB-MiniCast}}$ is quite imbalanced. As already mentioned, in symmetric applications, this imbalance is not a problem, but in asymmetric applications, it may be entirely unacceptable.

A second concern is that the “trick” we used to achieve the desired worst-case complexity bound actually introduces another type of imbalance, which may be problematic, even in symmetric applications. Each party sends its fragment to less than $n/3$ parties. This ensures that each party (other than S) transmits $\sim \frac{1}{2}\ell$ bits. However, there may well be parties that end up downloading fragments from *all* parties. For such “unlucky” parties, the total number of bits that they download can be as high as $\sim 2.5\ell$. Worse, this can happen even when all parties, even S , are honest. Even in symmetric applications, a party could be “unlucky” in all instances of the reliable broadcast protocol, and potentially saturate its download bandwidth. (The same issue arises in the protocols in [Loc24], which use the same “trick”).

One possible mitigation to this problem is to use an “on demand” strategy in which each party explicitly requests fragments from their peers. In addition, each party can choose to wait a certain amount of time after seeing a certified tag before making any requests, and only if it has not received the corresponding message directly from S by that time. (This idea was used in a similar setting by [Loc24].) This adds extra latency in the worst case, but not in the common case. The nice thing about this approach is that in the “happy case”, when S is honest and the network is well behaved, no honest party requests any fragments, and no fragments are sent by honest parties except to corrupt parties who request them just to waste bandwidth. In this case, if $t^* \leq t$ is the actual number of corrupt parties, the total communication complexity is $\sim \ell(n + 1.5t^*)$. In particular, if there are no faults (other than crash failures) and the network is well behaved, the total communication complexity is $\sim \ell n$, and the protocol adds essentially no communication overhead to the trivial (insecure) broadcast protocol in which S just sends its message to all parties.

The above “on demand” strategy is reminiscent of what is often done in practice in many consensus protocols (for example, [DKSS22]), where some form of simple “consistent broadcast” protocol is used, which does not guarantee completeness. To get around this lack of completeness, parties may request a missing message from other parties. However, in the worst case, this simple strategy can lead to communication complexity $\gg \ell n^2$, since all corrupt parties may request messages from all honest parties. Using our more refined approach, the total communication complexity is at most $\sim 1.5\ell n$, even in the worst case.

A third concern is that the protocol requires an accountable threshold signature scheme, which may result in larger signature certificates. We have already noted that if we use BLS-based aggregate signatures, the additional overhead is just n bits, which for values of n up to several hundred is not a practical concern. However, as all parties broadcast these signature certificates, a term $O(n^3)$ is added to the communication complexity, resulting in a worse (theoretical) bound with respect to n .

One could also consider a variant of $\Pi_{\text{UB-MiniCast}}$ in which we do not use an accountable threshold signature scheme, and each party sends its fragment to all other parties. For this variant, the worst-case communication complexity is $\sim 2.5\ell n$. However, when combined with the above “on demand” strategy, the communication complexity in the “happy case” is exactly the same as in the version that uses an accountable scheme.

References

- ADVZ22. N. Alhaddad, S. Duan, M. Varia, and H. Zhang. Practical and improved byzantine reliable broadcast and asynchronous verifiable information dispersal from hash functions. Cryptology ePrint Archive, Paper 2022/171, 2022. <https://eprint.iacr.org/2022/171>.
- ANRX21. I. Abraham, K. Nayak, L. Ren, and Z. Xiang. Good-case latency of byzantine broadcast: a complete categorization. In A. Miller, K. Censor-Hillel, and J. H. Korhonen, editors, *PODC '21: ACM Symposium*

- on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021, pages 331–341. ACM, 2021. <https://doi.org/10.1145/3465084.3467899>. Full version at <https://arxiv.org/abs/2102.07240>.
- ARV24. N. Alhaddad, L. Reyzin, and M. Varia. Committing AVID with partial retrieval and optimal storage. Cryptology ePrint Archive, Paper 2024/685, 2024. <https://eprint.iacr.org/2024/685>.
- BDN18. D. Boneh, M. Drijvers, and G. Neven. Compact multi-signatures for smaller blockchains. Cryptology ePrint Archive, Paper 2018/483, 2018. <https://eprint.iacr.org/2018/483>.
- BLS01. D. Boneh, B. Lynn, and H. Shacham. Short signatures from the Weil pairing. In C. Boyd, editor, *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings*, volume 2248 of *Lecture Notes in Computer Science*, pages 514–532. Springer, 2001. https://doi.org/10.1007/3-540-45682-1_30.
- Bol03. A. Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Y. Desmedt, editor, *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, volume 2567 of *Lecture Notes in Computer Science*, pages 31–46. Springer, 2003. https://doi.org/10.1007/3-540-36288-6_3.
- BPR24. D. Boneh, A. Partap, and L. Rotem. Proactive refresh for accountable threshold signatures. In J. Clark and E. Shi, editors, *Financial Cryptography and Data Security - 28th International Conference, FC 2024, Willemstad, Curaçao, March 4-8, 2024, Revised Selected Papers, Part II*, volume 14745 of *Lecture Notes in Computer Science*, pages 140–159. Springer, 2024. https://doi.org/10.1007/978-3-031-78679-2_8. Full version at <https://eprint.iacr.org/2022/1656>.
- Bra87. G. Bracha. Asynchronous byzantine agreement protocols. *Inf. Comput.*, 75(2):130–143, 1987. [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X).
- CDH⁺22. J. Camenisch, M. Drijvers, T. Hanke, Y. Pignolet, V. Shoup, and D. Williams. Internet computer consensus. In A. Milani and P. Woelfel, editors, *PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022*, pages 81–91. ACM, 2022. <https://doi.org/10.1145/3519270.3538430>. Full version at <https://eprint.iacr.org/2021/632>.
- CL99. M. Castro and B. Liskov. Practical byzantine fault tolerance. In M. I. Seltzer and P. J. Leach, editors, *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*, pages 173–186. USENIX Association, 1999.
- CP17. A. Choudhury and A. Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Trans. Inf. Theory*, 63(1):428–468, 2017. <https://doi.org/10.1109/TIT.2016.2614685>.
- CP23. B. Y. Chan and R. Pass. Simplex consensus: A simple and fast consensus protocol. Cryptology ePrint Archive, Paper 2023/463, 2023. <https://eprint.iacr.org/2023/463>.
- CT05. C. Cachin and S. Tessaro. Asynchronous verifiable information dispersal. In P. Fraigniaud, editor, *Distributed Computing, 19th International Conference, DISC 2005, Cracow, Poland, September 26-29, 2005, Proceedings*, volume 3724 of *Lecture Notes in Computer Science*, pages 503–504. Springer, 2005. https://doi.org/10.1007/11561927_42.
- DKSS22. G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In Y. Bromberg, A. Kermarrec, and C. Kozyrakis, editors, *EuroSys '22: Seventeenth European Conference on Computer Systems, Rennes, France, April 5 - 8, 2022*, pages 34–50. ACM, 2022. <https://doi.org/10.1145/3492321.3519594>. Also at arXiv:2105.11827, <https://arxiv.org/abs/2105.11827>.
- KKNS21. I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman. All you need is DAG. In A. Miller, K. Censor-Hillel, and J. H. Korhonen, editors, *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021*, pages 165–175. ACM, 2021. <https://doi.org/10.1145/3465084.3467905>. Also at arXiv:2102.08325, <https://arxiv.org/abs/2102.08325>.
- LAHC16. S. Lin, T. Y. Al-Naffouri, Y. S. Han, and W. Chung. Novel polynomial basis with fast Fourier transform and its application to Reed-Solomon erasure codes. *IEEE Trans. Inf. Theory*, 62(11):6284–6299, 2016. <https://doi.org/10.1109/TIT.2016.2608892>.
- LC12. S. Lin and W. Chung. An efficient (n, k) information dispersal algorithm for high code rate system over Fermat fields. *IEEE Commun. Lett.*, 16(12):2036–2039, 2012. <https://doi.org/10.1109/LCOMM.2012.112012.121322>.
- Loc24. T. Locher. Byzantine reliable broadcast with low communication and time complexity. In S. Bonomi, L. Galletta, E. Rivière, and V. Schiavoni, editors, *28th International Conference on Principles of Distributed Systems, OPODIS 2024, December 11-13, 2024, Lucca, Italy*, volume 324 of *LIPICs*, pages 16:1–

- 16:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. <https://doi.org/10.4230/LIPICS.OPODIS.2024.16>. Full version at <https://arxiv.org/abs/2404.08070>.
- LS25. T. Locher and V. Shoup. MiniCast: Minimizing the communication complexity of reliable broadcast. In S. Fehr and P. Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part V*, volume 15605 of *Lecture Notes in Computer Science*, pages 96–115. Springer, 2025. https://doi.org/10.1007/978-3-031-91092-0_4. Full version at <https://eprint.iacr.org/2024/571>.
- MXC⁺16. A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song. The honey badger of BFT protocols. In E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 31–42. ACM, 2016. <https://doi.org/10.1145/2976749.2978399>. Also at <https://eprint.iacr.org/2016/199>.
- Sho24. V. Shoup. Sing a song of simplex. In *38th International Symposium on Distributed Computing, DISC*, volume 319 of *LIPICs*, pages 37:1–37:22, 2024. <https://doi.org/10.4230/LIPICS.DISC.2024.37>. Full version at <https://eprint.iacr.org/2023/1916>.
- SS24. V. Shoup and N. P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *J. Cryptol.*, 37(3):27, 2024. <https://doi.org/10.1007/S00145-024-09505-6>. Also at <https://eprint.iacr.org/2023/536>.